LSTM for Time Series Prediction in PyTorch

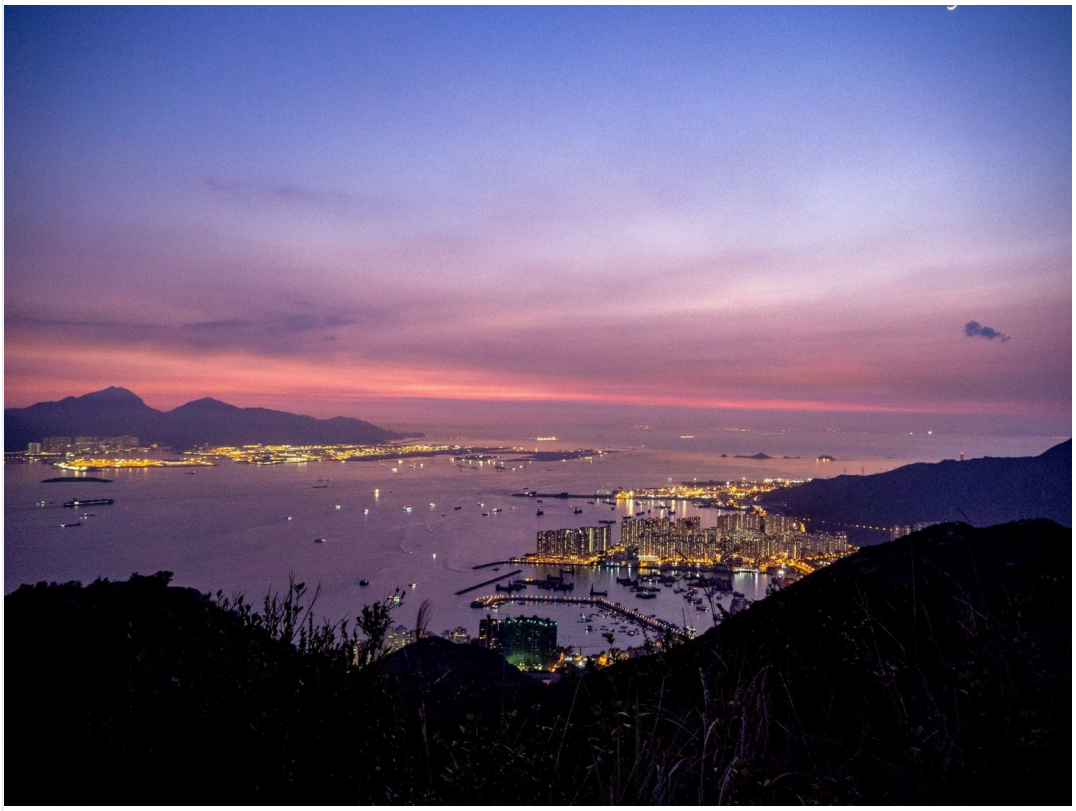
by **Adrian Tam** on April 8, 2023 in **Deep Learning with PyTorch** 56

Long Short-Term Memory (LSTM) is a structure that can be used in neural network. It is a type of recurrent neural network (RNN) that expects the input in the form of a sequence of features. It is useful for data such as time series or string of text. In this post, you will learn about LSTM networks. In particular,

- What is LSTM and how they are different
- How to develop LSTM network for time series prediction
- How to train a LSTM network

Kick-start your project with my book Deep Learning with PyTorch. It provides **self-study tutorials** with **working code**.

Let's get started.



LSTM for Time Series Prediction in PyTorch
Photo by Carry Kung. Some rights reserved.

Overview

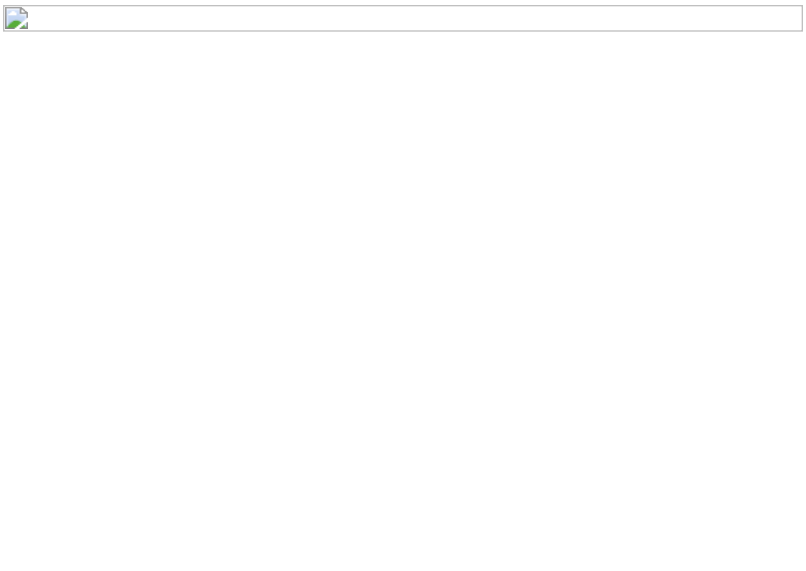
This post is divided into three parts; they are

- Overview of LSTM Network
- LSTM for Time Series Prediction
- Training and Verifying Your LSTM Network

Overview of LSTM Network

LSTM cell is a building block that you can use to build a larger neural network. While the common building block such as fully-connected layer are merely matrix multiplication of the weight tensor and the input to produce an output tensor, LSTM module is much more complex.

A typical LSTM cell is illustrated as follows



LSTM cell. Illustration from Wikipedia.

It takes one time step of an input tensor x as well as a cell memory c and a hidden state h. The cell memory and hidden state can be initialized to zero at the beginning. Then within the LSTM cell, x, c, and h will be multiplied by separate weight tensors and pass through some activation functions a few times. The result is the updated cell memory and hidden state. These updated c and h will be used on the **next time step** of the input tensor. Until the end of the last time step, the output of the LSTM cell will be its cell memory and hidden state.

Specifically, the equation of one LSTM cell is as follows:

$$
\begin{aligned}
f_t &= \sigma_g(W_f x_t + U_f h_{t-1} + b_f) \\
i_t &= \sigma_g(W_i x_t + U_i h_{t-1} + b_i) \\
o_t &= \sigma_g(W_o x_t + U_o h_{t-1} + b_o) \\
\tilde{c}_t &= \sigma_c(W_c x_t + U_c h_{t-1} + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
h_t &= o_t \odot \sigma_h(c_t)
\end{aligned}
$$

Where W, U, b are trainable parameters of the LSTM cell. Each equation above is computed for each time step, hence with subscript t. These trainable parameters are **reused** for all the time steps. This nature of shared parameter bring the memory power to the LSTM.

Note that the above is only one design of the LSTM. There are multiple variations in the literature.

Since the LSTM cell expects the input x in the form of multiple time steps, each input sample should be a 2D tensors: One dimension for time and another dimension for features. The power of an LSTM cell depends on the size of the hidden state or cell memory, which usually has a larger dimension than the number of features in the input.

LSTM for Time Series Prediction

Let's see how LSTM can be used to build a time series prediction neural network with an example.

The problem you will look at in this post is the international airline passengers prediction problem. This is a problem where, given a year and a month, the task is to predict the number of international airline passengers in units of 1,000. The data ranges from January 1949 to December 1960, or 12 years, with 144 observations.

It is a regression problem. That is, given the number of passengers (in unit of 1,000) the recent months, what is the number of passengers the next month. The dataset has only one feature: The number of passengers.

Let's start by reading the data. The data can be downloaded here.

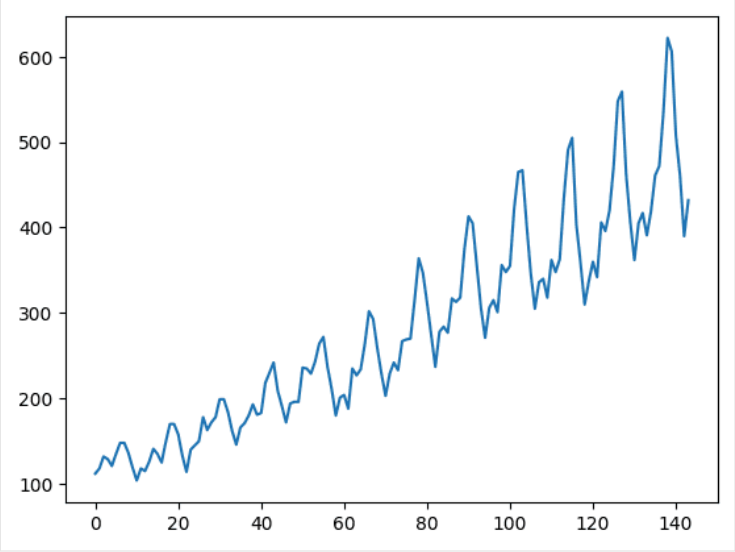
Save this file as `airline-passengers.csv` in the local directory for the following.

Below is a sample of the first few lines of the file:

```
"Month","Passengers"
"1949-01",112
"1949-02",118
"1949-03",132
"1949-04",129
```

The data has two columns, the month and the number of passengers. Since the data are arranged in chronological order, you can take only the number of passenger to make a single-feature time series. Below you will use pandas library to read the CSV file and convert it into a 2D numpy array, then plot it using matplotlib:

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv('airline-passengers.csv')
timeseries = df[["Passengers"]].values.astype('float32')

plt.plot(timeseries)
plt.show()
```



This time series has 144 time steps. You can see from the plot that there is an upward trend. There are also some periodicity in the dataset that corresponds to the summer holiday period in the northern hemisphere. Usually a time series should be "detrended" to remove the linear trend component and normalized before processing. For simplicity, these are skipped in this project.

To demonstrate the predictive power of our model, the time series is splitted into training and test sets. Unlike other dataset, usually time series data are splitted without shuffling. That is, the training set is the first half of time series and the remaining will be used as the test set. This can be easily done on a numpy array:

```
# train-test split for time series
train_size = int(len(timeseries) * 0.67)
test_size = len(timeseries) - train_size
train, test = timeseries[:train_size], timeseries[train_size:]
```

The more complicated problem is how do you want the network to predict the time series. Usually time series prediction is done on a window. That is, given data from time $t-w$ to time t, you are asked to predict for time $t+1$ (or deeper into the future). The size of window w governs how much data you are allowed to look at when you make the prediction. This is also called the **look back period**.

On a long enough time series, multiple overlapping window can be created. It is convenient to create a function to generate a dataset of fixed window from a time series. Since the data is going to be used in a PyTorch model, the output dataset should be in PyTorch tensors:

```
import torch

def create_dataset(dataset, lookback):
    """Transform a time series into a prediction dataset

    Args:
        dataset: A numpy array of time series, first dimension is the time steps
        lookback: Size of window for prediction
    """
    X, y = [], []
    for i in range(len(dataset)-lookback):
        feature = dataset[i:i+lookback]
        target = dataset[i+1:i+lookback+1]
        X.append(feature)
        y.append(target)
    return torch.tensor(X), torch.tensor(y)
```

This function is designed to apply windows on the time series. It is assumed to predict for one time step into the immediate future. It is designed to convert a time series into a tensor of dimensions (window sample, time steps, features). A time series of L time steps can produce roughly L windows (because a window can start from any time step as long as the window does not go beyond the boundary of the time series). Within one window, there are multiple consecutive time steps of values. In each time step, there can be multiple features. In this dataset, there is only one.

It is intentional to produce the "feature" and the "target" the same shape: For a window of three time steps, the "feature" is the time series from t to $t+2$ and the target is from $t+1$ to $t+3$. What we are interested is $t+3$ but the information of $t+1$ to $t+2$ is useful in training.

Note that the input time series is a 2D array and the output from the `create_dataset()` function will be a 3D tensors. Let's try with `lookback=1`. You can verify the shape of the output tensor as follows:

```
lookback = 1
X_train, y_train = create_dataset(train, lookback=lookback)
X_test, y_test = create_dataset(test, lookback=lookback)
print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)
```

which you should see:

```
torch.Size([95, 1, 1]) torch.Size([95, 1, 1])
torch.Size([47, 1, 1]) torch.Size([47, 1, 1])
```

Now you can build the LSTM model to predict the time series. With `lookback=1`, it is quite surely that the accuracy would not be good for too little clues to predict. But this is a good example to demonstrate the structure of the LSTM model.

The model is created as a class, in which a LSTM layer and a fully-connected layer is used.

```
...
import torch.nn as nn

class AirModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=50, num_layers=1, batch_first=True)
        self.linear = nn.Linear(50, 1)
    def forward(self, x):
        x, _ = self.lstm(x)
        x = self.linear(x)
        return x
```

The output of `nn.LSTM()` is a tuple. The first element is the generated hidden states, one for each time step of the input. The second element is the LSTM cell's memory and hidden states, which is not used here.

The LSTM layer is created with option `batch_first=True` because the tensors you prepared is in the dimension of (window sample, time steps, features) and where a batch is created by sampling on the first dimension.

The output of hidden states is further processed by a fully-connected layer to produce a single regression result. Since the output from LSTM is one per each input time step, you can chooce to pick only the last timestep's output, which you should have:

```
x, _ = self.lstm(x)
# extract only the last time step
x = x[:, -1, :]
x = self.linear(x)
```

and the model's output will be the prediction of the next time step. But here, the fully connected layer is applied to each time step. In this design, you should extract only the last time step from the model output as your prediction. However, in this case, the window is 1, there is no difference in these two approach.

Training and Verifying Your LSTM Network

Because it is a regression problem, MSE is chosen as the loss function, which is to be minimized by Adam optimizer. In the code below, the PyTorch tensors are combined into a dataset using `torch.utils.data.TensorDataset()` and batch for training is provided by a `DataLoader`. The model performance is evaluated once per 100 epochs, on both the trainning set and the test set:

```
import numpy as np
import torch.optim as optim
import torch.utils.data as data

model = AirModel()
optimizer = optim.Adam(model.parameters())
loss_fn = nn.MSELoss()
loader = data.DataLoader(data.TensorDataset(X_train, y_train), shuffle=True, batch_size=8)

n_epochs = 2000
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    if epoch % 100 != 0:
        continue
    model.eval()
    with torch.no_grad():
        y_pred = model(X_train)
        train_rmse = np.sqrt(loss_fn(y_pred, y_train))
        y_pred = model(X_test)
        test_rmse = np.sqrt(loss_fn(y_pred, y_test))
    print("Epoch %d: train RMSE %.4f, test RMSE %.4f" % (epoch, train_rmse, test_rmse))
```

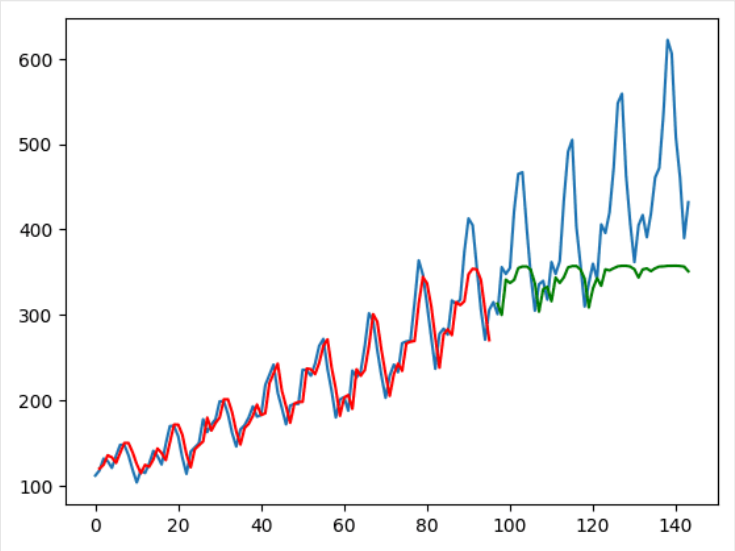
As the dataset is small, the model should be trained for long enough to learn about the pattern. Over these 2000 epochs trained, you should see the RMSE on both training set and test set decreasing:

```
Epoch 0: train RMSE 225.7571, test RMSE 422.1521
Epoch 100: train RMSE 186.7353, test RMSE 381.3285
Epoch 200: train RMSE 153.3157, test RMSE 345.3290
Epoch 300: train RMSE 124.7137, test RMSE 312.8820
Epoch 400: train RMSE 101.3789, test RMSE 283.7040
Epoch 500: train RMSE 83.0900, test RMSE 257.5325
Epoch 600: train RMSE 66.6143, test RMSE 232.3288
Epoch 700: train RMSE 53.8428, test RMSE 209.1579
Epoch 800: train RMSE 44.4156, test RMSE 188.3802
Epoch 900: train RMSE 37.1839, test RMSE 170.3186
Epoch 1000: train RMSE 32.0921, test RMSE 154.4092
Epoch 1100: train RMSE 29.0402, test RMSE 141.6920
Epoch 1200: train RMSE 26.9721, test RMSE 131.0108
Epoch 1300: train RMSE 25.7398, test RMSE 123.2518
Epoch 1400: train RMSE 24.8011, test RMSE 116.7029
Epoch 1500: train RMSE 24.7705, test RMSE 112.1551
Epoch 1600: train RMSE 24.4654, test RMSE 108.1879
Epoch 1700: train RMSE 25.1378, test RMSE 105.8224
Epoch 1800: train RMSE 24.1940, test RMSE 101.4219
Epoch 1900: train RMSE 23.4605, test RMSE 100.1780
```

It is expected to see the RMSE of test set is an order of magnitude larger. The RMSE of 100 means the prediction and the actual target would be in average off by 100 in value (i.e., 100,000 passengers in this dataset).

To better understand the prediction quality, you can indeed plot the output using matplotlib, as follows:

```
with torch.no_grad():
    # shift train predictions for plotting
    train_plot = np.ones_like(timeseries) * np.nan
    y_pred = model(X_train)
    y_pred = y_pred[:, -1, :]
    train_plot[lookback:train_size] = model(X_train)[:, -1, :]
    # shift test predictions for plotting
    test_plot = np.ones_like(timeseries) * np.nan
    test_plot[train_size+lookback:len(timeseries)] = model(X_test)[:, -1, :]
# plot
plt.plot(timeseries, c='b')
plt.plot(train_plot, c='r')
plt.plot(test_plot, c='g')
plt.show()
```



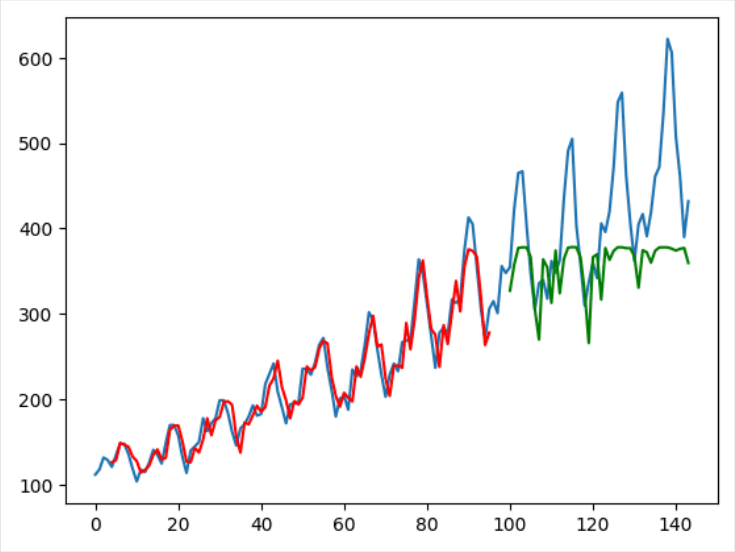
From the above, you take the model's output as `y_pred` but extract only the data from the last time step as `y_pred[:, -1, :]`. This is what is plotted on the chart.

The training set is plotted in red while the test set is plotted in green. The blue curve is what the actual data looks like. You can see that the model can fit well to the training set but not very well on the test set.

Tying together, below is the complete code, except the parameter `lookback` is set to 4 this time:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data

df = pd.read_csv('airline-passengers.csv')
timeseries = df[["Passengers"]].values.astype('float32')

# train-test split for time series
train_size = int(len(timeseries) * 0.67)
test_size = len(timeseries) - train_size
train, test = timeseries[:train_size], timeseries[train_size:]

def create_dataset(dataset, lookback):
    """Transform a time series into a prediction dataset

    Args:
        dataset: A numpy array of time series, first dimension is the time steps
        lookback: Size of window for prediction
    """
    X, y = [], []
    for i in range(len(dataset)-lookback):
        feature = dataset[i:i+lookback]
        target = dataset[i+1:i+lookback+1]
        X.append(feature)
        y.append(target)
    return torch.tensor(X), torch.tensor(y)

lookback = 4
X_train, y_train = create_dataset(train, lookback=lookback)
X_test, y_test = create_dataset(test, lookback=lookback)

class AirModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=50, num_layers=1, batch_first=True)
        self.linear = nn.Linear(50, 1)
    def forward(self, x):
        x, _ = self.lstm(x)
        x = self.linear(x)
        return x

model = AirModel()
optimizer = optim.Adam(model.parameters())
loss_fn = nn.MSELoss()
loader = data.DataLoader(data.TensorDataset(X_train, y_train), shuffle=True, batch_size=8)

n_epochs = 2000
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
```

```
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    if epoch % 100 != 0:
        continue
    model.eval()
    with torch.no_grad():
        y_pred = model(X_train)
        train_rmse = np.sqrt(loss_fn(y_pred, y_train))
        y_pred = model(X_test)
        test_rmse = np.sqrt(loss_fn(y_pred, y_test))
    print("Epoch %d: train RMSE %.4f, test RMSE %.4f" % (epoch, train_rmse, test_rmse))

with torch.no_grad():
    # shift train predictions for plotting
    train_plot = np.ones_like(timeseries) * np.nan
    y_pred = model(X_train)
    y_pred = y_pred[:, -1, :]
    train_plot[lookback:train_size] = model(X_train)[:, -1, :]
    # shift test predictions for plotting
    test_plot = np.ones_like(timeseries) * np.nan
    test_plot[train_size+lookback:len(timeseries)] = model(X_test)[:, -1, :]
# plot
plt.plot(timeseries)
plt.plot(train_plot, c='r')
plt.plot(test_plot, c='g')
plt.show()
```



Running the above code will produce the plot below. From both the RMSE measure printed and the plot, you can notice that the model can now do better on the test set.

This is also why the `create_dataset()` function is designed in such way: When the model is given a time series of time t to $t+3$ (as `lookback=4`), its output is the prediction of $t+1$ to $t+4$. However, $t+1$ to $t+3$ are also known from the input. By using these in the loss function, the model effectively was provided with more clues to train. This design is not always suitable but you can see it is helpful in this particular example.

Further Readings

This section provides more resources on the topic if you are looking to go deeper.

- `nn.LSTM()` from PyTorch documentation
- `torch.utils.data` API from PyTorch

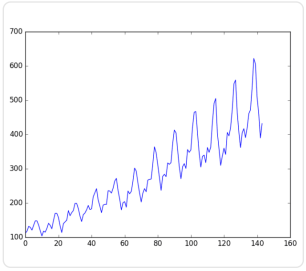
Summary

In this post, you discovered what is LSTM and how to use it for time series prediction in PyTorch. Specifically, you learned:

- What is the international airline passenger time series prediction dataset
- What is a LSTM cell
- How to create an LSTM network for time series prediction

More On This Topic

Time Series Prediction with LSTM Recurrent Neural…

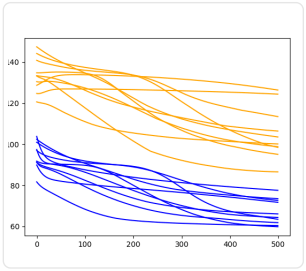


Time Series Prediction with Deep Learning in Keras

Understand Time Series Forecast Uncertainty Using…

Using CNN for financial time series prediction

Time Series Forecasting with PyCaret: Building…



How to Tune LSTM Hyperparameters with Keras for Time…

About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
View all posts by Adrian Tam →

‹ Handwritten Digit Recognition with LeNet5 Model in PyTorch

Text Generation with LSTM in PyTorch ›

56 Responses to *LSTM for Time Series Prediction in PyTorch*

eliasnemo March 16, 2023 at 3:24 am #

REPLY

Forgive me, there must be an error somewhere:
print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)
torch.Size([95, 1]) torch.Size([95, 1])
torch.Size([47, 1]) torch.Size([47, 1])

James Carmichael March 16, 2023 at 7:05 am #

REPLY

Hi Eliasnemo…What is the error you are referring to?

eliasnemo March 17, 2023 at 9:58 am #

REPLY

Hi James, I apologize I wrote the comment too hastily, the error was mine, the shape of my timeseries was (432,) while yours is (432,1) this generated in the create_dataset(dataset, lookback) function an incorrect tensor shape: torch.Size([95, 1]) torch.Size([95, 1]) torch.Size([47, 1]) torch.Size([47, 1]) and not the correct one: torch.Size([95, 1, 1]) torch.Size([95, 1, 1]) torch.Size([47, 1, 1]) torch.Size([47, 1, 1]) as in your example. I just added X=np.array(X) and y np.array(y) before return torch.tensor(X), torch.tensor(y) to avoid the message “UserWarning: Creating a tensor from a list of numpy.ndarrays is extremely slow…”. Thank you for sharing your excellent work.

tqrahman March 23, 2023 at 3:22 pm #

REPLY

Hi, this post is informative! In line 65-68, should it be X_batch and y_batch instead of X_train and y_train?

James Carmichael March 24, 2023 at 6:09 am #

REPLY

Hi tqrahman…I do not see an error. What is your results by changing the code to that for which you suggested?

Bot March 4, 2025 at 2:24 pm #

REPLY

Trading bot siganls

James Carmichael March 5, 2025 at 9:31 am #

REPLY

Thank you Bot! No bots in place in this forum!

James Carmichael March 5, 2025 at 9:32 am #

And you the spelling is “signals” 🙂

guest April 9, 2023 at 11:37 pm #

REPLY

do you have example for predict next x day graph plot

James Carmichael April 10, 2023 at 8:09 am #

REPLY

Hi guest…please clarify your question so that we may better assist you.

Sobiedd April 18, 2023 at 9:09 pm #

REPLY

Hi James, I'm wondering, if we use the "create_dataset" function to create the windows for the training, after training the model and using it ti predict we will need to transform our new dataset and have the same shape to predict, therefore, we won't predict for the last "N lookback" instances due to the function is only getting windows for "len(dataset)-lookback". In conclusion, we won't predict values for the whole dataset, if I use lookback=3, I won't get predictions for the last 3 instances.

James Carmichael April 19, 2023 at 9:35 am # REPLY

Hi Sobiedd…I am not certain I am following your question. Have you performed a prediction and have noted an issue with the suggested methodology used in the tutorial? Perhaps we can start with your results and determine if there is something missing from the implementation.

Daneshwari May 16, 2023 at 4:13 pm # REPLY

Hi Jason! Thanks for this blog, it's really helpful. I intended to use the lstm network for prediction. The dataset I have is a social media dataset with multiple variables(image features, posts posting date, tags, location info etc.). This dataset has temporal features, so I can plot each post v/s the output, taking month-year(the time scale) on x-axis and o/p on y-axis.

I have done the feature engineering and now I wanted to train a lstm model to predict the output. But since NN/lstm models need data to be normalized, I was wondering –
1.) whether to normalize/scale the data,
2.) should I normalize considering each feature for each samples? or should I normalize it feature-wise(normalize by tags_length feature/column)?

I need your suggestion as early as possible since I'm aiming for a deadline. Any help is highly appreciated. I look forward to your suggestion. Thank You!

Angelo June 20, 2023 at 6:03 am # REPLY

Hi James,

Perhaps I am wrong, but it seems that you are using teacher forcing during the test phase. It doesn't seem like the model is autoregressive. I would like to see results where the LSTM uses its own predictions to generate new ones.

James Carmichael June 20, 2023 at 9:57 am # REPLY

Hi Angelo…The following example may help add clarity:

https://machinelearningmastery.com/how-to-develop-lstm-models-for-multi-step-time-series-forecasting-of-household-power-consumption/

Santobedi July 17, 2023 at 11:44 pm # REPLY

Hi James,

If I have to predict several consecutive future time-series values, how can I do it? For example, if I have predictor variables and target till now (t=0), how can I predict the target at t+1, t+2, and t+3? In other words, if I have the predictor variables and target for every hour (as historical data), how can I predict the values of the target for the upcoming three hours? How can I prepare the dataset and update my model (e.g., LSTM)?

Thank you.

Tom September 1, 2023 at 7:51 pm # REPLY

Hi James,

I'm just checking – in the snipper where you extract the last step, are we missing a colon after the "-1"?

Something like

x = x[:, -1:, :]

James Carmichael September 2, 2023 at 8:15 am # REPLY

Hi Tom…Thank you for your feedback! We do not see an issue with the original code. What did you find when you executed it?

Myk October 5, 2023 at 7:31 am #

REPLY

Given time series nature of the input, why did you set shuffle=true in DataLoader?

James Carmichael October 5, 2023 at 9:51 am #

REPLY

Hi Myk…The following resource may be of interest:

https://machinelearningmastery.com/training-a-pytorch-model-with-dataloader-and-dataset/#:~:text=The%20batch%20size%20is%20a,to%20read%20every%20batch%20once.

Myk October 6, 2023 at 1:16 am #

REPLY

The samples may be shuffled because each sample is independent. That is, a given sample captures an entire input sequence; therefore, sequential information is retained even as samples are shuffled.

Myk October 6, 2023 at 5:34 am #

REPLY

In your “complete code” above, lines 74-75 are redundant, as line 76 does the same thing:

```
y_pred = model(X_train)
y_pred = y_pred[:, -1, :]
train_plot[lookback:train_size] = model(X_train)[:, -1, :]
```

Majk October 19, 2023 at 1:01 am #

REPLY

Both graphs seem to be shifted in both train and test plot. Why is that happening? And how to fix it?

James Carmichael October 19, 2023 at 9:07 am #

REPLY

You are working on a time series forecasting problem and you plot your forecasted time series against the actual time series and it looks like the forecast is one step behind the actual.

This is common.

It means that your model is making a persistence forecast. This is a forecast where the input to the forecast (e.g. the observation at the previous time step) is predicted as the output.

The persistence forecast is used as a baseline method for comparison on time series forecasting. You can learn more about the method here:

How to Make Baseline Predictions for Time Series Forecasting with Python
The persistence forecast is the best that we can do on challenging time series forecasting problems, such as those series that are a random walk, like short range movements of stock prices. You can learn more about this here:

A Gentle Introduction to the Random Walk for Times Series Forecasting with Python
If your sophisticated model, such as a neural network, is outputting a persistence forecast, it might mean:

That the model requires further tuning.
That the chosen model cannot address your specific dataset.
It might also mean that your time series problem is not predictable.

Saranga November 11, 2023 at 4:44 pm #

REPLY

Hi there,
All those tutorials refer to forecast training and testing, but can you do a one which actually forecast beyond the dataset as example next 3 .months forecast

James Carmichael November 12, 2023 at 10:31 am #

REPLY

Hi Saranga…The following resource may be of interest to you:

https://stackoverflow.com/questions/69906416/forecast-future-values-with-lstm-in-python

Murilo November 12, 2023 at 12:37 am #

REPLY

Does LSTM works for time series classification?

Michael December 1, 2023 at 2:49 pm #

REPLY

Looking at your final graph above, it appears that the trained model is still only doing a persistence forecast as it is almost an exact shifted version of the dataset. Is that a reflection of an issue in this lookback approach implementation in general or is it a reflection of the lack of useful features in the dataset? How would you proceed from here to make it more accurate?

James Carmichael December 2, 2023 at 11:35 am #

REPLY

Hi Michael…You are working on a time series forecasting problem and you plot your forecasted time series against the actual time series and it looks like the forecast is one step behind the actual.

This is common.

It means that your model is making a persistence forecast. This is a forecast where the input to the forecast (e.g. the observation at the previous time step) is predicted as the output.

The persistence forecast is used as a baseline method for comparison on time series forecasting. You can learn more about the method here:

How to Make Baseline Predictions for Time Series Forecasting with Python
The persistence forecast is the best that we can do on challenging time series forecasting problems, such as those series that are a random walk, like short range movements of stock prices. You can learn more about this here:

A Gentle Introduction to the Random Walk for Times Series Forecasting with Python
If your sophisticated model, such as a neural network, is outputting a persistence forecast, it might mean:

That the model requires further tuning.
That the chosen model cannot address your specific dataset.
It might also mean that your time series problem is not predictable.

Michael December 2, 2023 at 1:47 pm #

REPLY

I played a bit with your code and I note that if I adjust the settings to eliminate the persistence shift on the trained set by increasing lookback or network size, I end up over fitting and getting even worse performance on the test set, so I get the impression this is a hard tradeoff in the case of this lookback lstm stackup. I read the other material, it is helpful, but doesn't show me a way out of this tradeoff of either overfitting or persistence.

Murilo December 22, 2023 at 4:23 am #

REPLY

I have a few questions:

1 – Is 'hidden_size' from Pytorch the same parameter as 'units' in Tensorflow/Keras?

2 – If yes, what they actually represent in the first figure in this post? For example, if we have 'hidden_size = X', we have X LSTM cells? Or when we define 'nn.LSTM(input_size=1, hidden_size=X, num_layers=1, batch_first=True)' we have one cell no matter the value of X?

3 – And how they are conected to the linear/dense layer? Each cell is conected to each node in the linear layer? Or just the last cell is conected to each node?

Alexander February 12, 2024 at 7:52 am #

REPLY

Is the next line correct ?
target = dataset[i+1:i+lookback+1]

Because we want as target the next direct output, it should be:
target = dataset[i+lookback:i+lookback+1]

James Carmichael February 12, 2024 at 8:15 am #

REPLY

Hi Alexander…It should be correct. Did you execute the code? If so what did you find?

J April 5, 2024 at 8:55 pm #

REPLY

Hi. I have a question to confirm my observation. Does the number of records on both training and test splits lessened based on the number of lookbacks? For instance in my case, there were 699 records for the original train split which became 698 after applying create_dataset() function. The same happened for my test wherein from 48, it became 47.

The same length was also applied for train and test predictions. I also have another question with regards to displaying the plot. Since the number of train and test predictions are different from the number of records from the original train and test splits, how should I plot it with the x-axis as the datetime stamp?

James Carmichael April 7, 2024 at 7:18 am #

REPLY

Hi J…Based on your description, it sounds like you are observing a common behavior in time series data preparation when using a function like `create_dataset()` which typically is used to reformat a time series dataset into a format suitable for LSTM models, by creating lookback sequences. Let's clarify and answer both parts of your question:

Reduction in Records Due to Lookbacks

The reduction in the number of records from your original dataset to what you have after applying the `create_dataset()` function is indeed expected due to the nature of lookback processing. Here's how it works:

– **Lookback Logic**: If you are using a lookback period (also known as lag, window size, or sequence length), the function needs to create sequences of that many past observations to predict the current value. For example, with a lookback of 1, each input sequence for your model will consist of one previous time step to predict the current time step.

– **Effect on Data Size**: This means that the first few records in your dataset (exactly as many as your lookback period) won't have enough previous data points to form a complete sequence. Thus, these records are typically dropped from the training or testing datasets. For a lookback of 1, you lose 1 data point at the start, which matches what you observed: 699 records becoming 698, and 48 becoming 47.

Plotting Data with Mismatched Lengths

Regarding plotting the training and test predictions alongside the original data with timestamps on the x-axis, considering the mismatch in lengths due to the lookback, you can handle this by adjusting the index of your predictions. Here's how you can do it:

1. **Offset Adjustments**: Since each prediction corresponds to an output where the input sequence ends, you should start plotting predictions from the index equivalent to the lookback period. For example, if your lookback is 1, your predictions should start from the second record in your original dataset.

2. **Code Example**: Suppose your DataFrame with the original time series is `df`, and it includes a datetime column `date`. Here's a basic plotting approach using Python and matplotlib:

```
python
import matplotlib.pyplot as plt

# Sample data
dates = df['date'] # Assuming 'date' is your datetime column
original_train = df['value'][:698] # Assuming 'value' is what you're predicting
original_test = df['value'][698:]

# Assuming train_predictions and test_predictions are your model outputs
train_predictions = [None] + list(train_predictions) # Offset for alignment
test_predictions = [None] * 699 + list(test_predictions) # Offset for alignment

plt.figure(figsize=(15, 8))
plt.plot(dates, df['value'], label='Original Data')
plt.plot(dates, train_predictions, label='Train Predictions')
plt.plot(dates, test_predictions, label='Test Predictions')
plt.legend()
plt.title('Time Series Prediction')
plt.xlabel('Date')
plt.ylabel('Value')
plt.show()
```

Key Points in the Plotting Code:

– **Alignment by Index**: The `None` values are added to the predictions list to align the predictions correctly with the original data indices. Adjust the number of `None` values based on your exact lookback and how your splits are structured.
– **Plotting All Together**: This script plots the original data along with the adjusted predictions on the same graph for visual comparison.

This approach will help you visually compare how well your model's predictions match up against the actual values, taking into account the datetime sequence. Adjust the plotting details as needed to fit your specific setup and visualization needs.

Half Dome May 5, 2024 at 9:35 am #

REPLY

If we add a single line logging the time series in the first cell:

timeseries = np.log(timeseries)

We get an almost perfect fit against the test set.

James Carmichael May 6, 2024 at 7:11 am #

REPLY

Hi Half Dome…Thank you for your feedback and suggestion!

Jack chen September 3, 2024 at 5:34 pm #

REPLY

Hello, I have a question, does the standard LSTM include teacher forcing? Do you consider teacher forcing in your code?

James Carmichael September 4, 2024 at 7:48 am #

REPLY

Hi Jack…The standard LSTM (Long Short-Term Memory) architecture does not inherently include teacher forcing. Teacher forcing is a training technique often used in sequence-to-sequence models, particularly in tasks like language translation, where the output of the model at a previous time step is fed as input for the next time step during training.

Understanding Teacher Forcing:
– **Teacher Forcing**: During training, instead of using the model's predicted output from the previous time step as the input for the next step, the actual ground truth (the correct previous output) is used. This can help the model learn faster and improve stability during training, particularly in the early stages.

Implementing Teacher Forcing:
If you want to use teacher forcing with an LSTM in your code, you will need to implement it manually. Here's a simple way to include teacher forcing in an LSTM-based model using PyTorch:

```
python
import torch
import torch.nn as nn
```

class LSTMModel(nn.Module):
def __init__(self, input_size, hidden_size, output_size, num_layers=1):
super(LSTMModel, self).__init__()
self.lstm = nn.LSTM(input_size, hidden_size, num_layers)
self.fc = nn.Linear(hidden_size, output_size)

def forward(self, input_seq, target_seq=None, teacher_forcing_ratio=0.5):
# input_seq: shape (seq_len, batch_size, input_size)
# target_seq: shape (seq_len, batch_size, output_size)
outputs = []
hidden, cell = None, None

# Loop through the sequence
for t in range(input_seq.size(0)):
if t == 0 or target_seq is None or torch.rand(1).item() > teacher_forcing_ratio:
input_t = input_seq[t].unsqueeze(0) # use the predicted value
else:
input_t = target_seq[t-1].unsqueeze(0) # use the actual target

output, (hidden, cell) = self.lstm(input_t, (hidden, cell) if hidden is not None else None)
output = self.fc(output)
outputs.append(output)

outputs = torch.cat(outputs, dim=0) # Concatenate outputs across the time dimension
return outputs

```
# Example usage:
# lstm_model = LSTMModel(input_size, hidden_size, output_size)
# output = lstm_model(input_seq, target_seq, teacher_forcing_ratio=0.5)
```

Key Points:
– **Teacher Forcing Ratio**: You can control how often teacher forcing is applied with the `teacher_forcing_ratio`. A ratio of 1.0 means always

using the true previous output, while 0.0 means never using it (only using the model's predictions).
– **Flexibility**: This approach gives you the flexibility to experiment with and without teacher forcing to see which works best for your specific task.

Prommin V September 8, 2024 at 12:15 am #

REPLY

Thank you very much for providing this helpful tutorial. I have some questions regarding the plotting section.

I understand that we are using the prediction for only the last value of each window. However, in the plotting section, why are the prediction values placed continuously, instead of placing them with lookback step interval.

James Carmichael September 8, 2024 at 7:30 am #

REPLY

Hi Prommin…In LSTM-based time series prediction, while you're using a specific window size (or lookback) to make predictions, the continuous placement of predictions in the plot is likely due to how the model is evaluated and how the results are plotted.

Here's why this might happen:

1. **Sliding Window Prediction**:
When training and predicting using a sliding window, the LSTM model takes a sequence of past data points (the lookback window) and predicts the next value. After making this prediction, the window is shifted by one time step, and the next sequence is used to predict the subsequent value. This process continues, resulting in predictions being made at every time step, even though each prediction is based on a sliding window of previous steps.

2. **Plotting Predictions Continuously**:
During plotting, since the model makes predictions at each time step, these predicted values are typically plotted continuously, even though each prediction corresponds to the end of a window. This gives the appearance of a continuous line in the plot because you are essentially predicting one time step ahead for each sliding window.

3. **Lookback Interval vs. Predictions**:
If you were to only plot the predictions at intervals corresponding to the lookback window, you'd only have predictions every few steps (depending on your lookback size). However, this would make the prediction plot sparse, and you wouldn't see a smooth curve of predictions. In many cases, continuous plotting gives a better visual representation of how well the model is performing across all time steps.

Adjusting Plotting Based on Lookback:
If you want to plot only the predictions made at specific intervals (e.g., only at the end of each lookback window), you could modify your plotting code to only show those points. This would involve keeping track of the indices where predictions are made and plotting only those.

In summary, the continuous placement of predictions in the plot comes from the model making one-step-ahead predictions at each time step, which is why the predictions are plotted without gaps. If you want the predictions to be plotted at the lookback step interval, you'll need to manually adjust the plotting logic.

Louis September 15, 2024 at 11:42 pm #

REPLY

Hi James

I just have a silly question in regards to the training device, how does one cast this onto a GPU rather than the CPU?

Thank you

James Carmichael September 16, 2024 at 3:36 am #

REPLY

Hi Louis…Hello,

That's a great question, and it's not silly at all! Running your PyTorch models on a GPU can significantly speed up training, especially for models like LSTMs that can be computationally intensive.

To cast your model and data to the GPU in PyTorch, you'll need to follow these steps:

1. **Check if a GPU is available**:
First, you need to check if CUDA (the computing platform for NVIDIA GPUs) is available on your system.

```
python
import torch

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)
```

This code sets the `device` to `'cuda'` if a GPU is available; otherwise, it defaults to `'cpu'`.

2. **Move your model to the GPU**:
After defining your model, move it to the GPU using the `.to()` method.

```
python
model = YourModelClass(*args)
model.to(device)
```

3. **Move your data to the GPU**:
When you load your data, you'll need to move the tensors to the GPU as well. This is typically done within your training loop.

```
python
for inputs, labels in data_loader:
inputs = inputs.to(device)
labels = labels.to(device)

# Now proceed with your forward and backward passes
outputs = model(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
```

Ensure that every tensor involved in computation is moved to the GPU. This includes inputs, labels, and any other tensors you might be using.

4. **Handle the hidden state (for RNNs like LSTM)**:
If your LSTM model maintains a hidden state that you initialize manually, make sure to move it to the GPU as well.

```
python
h0 = torch.zeros(num_layers, batch_size, hidden_size).to(device)
c0 = torch.zeros(num_layers, batch_size, hidden_size).to(device)
```

5. **Adjust your data types if necessary**:
Sometimes, you might need to ensure that your data types match between the model and inputs, especially when working with GPUs.

6. **Example Putting It All Together**:

```
python
import torch
import torch.nn as nn
import torch.optim as optim

# Define your device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)

# Define your model
class LSTMModel(nn.Module):
def __init__(self, input_size, hidden_size, num_layers, output_size):
super(LSTMModel, self).__init__()
self.hidden_size = hidden_size
self.num_layers = num_layers
self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
self.fc = nn.Linear(hidden_size, output_size)

def forward(self, x):
h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(device)
c0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(device)

out, _ = self.lstm(x, (h0, c0))
out = self.fc(out[:, -1, :])
return out

# Instantiate the model, define loss and optimizer
model = LSTMModel(input_size, hidden_size, num_layers, output_size).to(device)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# Training loop
for epoch in range(num_epochs):
for inputs, labels in data_loader:
inputs = inputs.to(device)
labels = labels.to(device)

# Forward pass
outputs = model(inputs)
loss = criterion(outputs, labels)

# Backward and optimize
optimizer.zero_grad()
loss.backward()
optimizer.step()

print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')
```

7. **Verify that CUDA is being used**:
You can verify that your tensors are on the GPU by checking their `.device` attribute.

```
python
print(next(model.parameters()).device)
```

This should output `'cuda:0'` if the model is on the GPU.

8. **Common Pitfalls**:

– **Forgetting to move data to the GPU**: Remember that both your model and your data need to be on the same device.
– **Inconsistent devices**: Ensure that any new tensors created during training are also moved to the GPU.
– **GPU Memory Limitations**: GPUs have limited memory. If you encounter out-of-memory errors, you may need to reduce your batch size.

9. **Additional Tips**:

– **DataLoader Optimization**: When using `DataLoader`, you can set `pin_memory=True` to speed up data transfer to GPU.

```
python
data_loader = DataLoader(dataset, batch_size=batch_size, shuffle=True, pin_memory=True)
```

– **Using Multiple GPUs**: If you have multiple GPUs, you can leverage `torch.nn.DataParallel` or `torch.nn.parallel.DistributedDataParallel` for parallel training.

```
python
if torch.cuda.device_count() > 1:
print(f"Using {torch.cuda.device_count()} GPUs")
model = nn.DataParallel(model)
model.to(device)
```

10. **Check GPU Utilization**:

– You can monitor your GPU usage with the `nvidia-smi` command in the terminal.

```
bash
watch -n 1 nvidia-smi
```

– This will help you ensure that your program is utilizing the GPU as expected.

—

By following these steps, you should be able to cast your model and training process onto a GPU. Running your training on a GPU can significantly reduce training time and handle larger models or datasets more efficiently.

Feel free to ask if you have any more questions or need further clarification!

Louis September 18, 2024 at 9:55 pm #

REPLY

Thank you very much!

If I can ask one more question, albeit a code/logic heavy one:

How would you go about implementing a validation step?

Nicole September 22, 2024 at 1:01 am #

REPLY

Good day James

I hope you are well.
I have a rather dumb question, I have checked online and keep getting errors. Is there a way one can add a r^2 score?
Thank you and have a lovely day

James Carmichael September 22, 2024 at 6:18 am #

REPLY

Hi Nicole…It's not a dumb question at all! Adding the R^2 score to evaluate your LSTM model's performance is actually quite useful.

Here's how you can compute the R^2 score in PyTorch:

1. **Import the required module**:
```
python
from sklearn.metrics import r2_score
```

2. **Calculate the R^2 score**:
After you've made predictions with your LSTM model, you can compare the predicted values with the true target values (both should be of the same shape). For example:

```
python
# Assuming y_true is your true values and y_pred is your model's predicted values
y_true = y_true.detach().cpu().numpy() # Convert tensors to numpy arrays if they are in tensor form
y_pred = y_pred.detach().cpu().numpy()

r2 = r2_score(y_true, y_pred)
print(f'R^2 Score: {r2}')
```

Make sure that both `y_true` and `y_pred` are either tensors or numpy arrays, and they need to be in the same shape before passing them to `r2_score`.

Common Errors to Avoid:
– **Shape mismatch**: Ensure the shapes of `y_true` and `y_pred` are identical.
– **Tensor to Numpy conversion**: If your data is in the form of PyTorch tensors, use `.detach().cpu().numpy()` to convert them to numpy arrays for compatibility with `r2_score`.

Let me know if you run into any errors, and I'd be happy to help! Have a lovely day too!

Nicole September 25, 2024 at 9:41 pm #

REPLY

Good day

Thank you so much for the help. And sorry to bother again. I have little experience with modelling so am troubleshooting one thing at a time. So based on my understanding, the prediction is model(X_test) and then the real should be y_test. As they are split in the train_test_split and after in the create dataset, they should be the same size? I'm getting errors that they aren't, how can I fix this?
Thank you so so much again
Have a lovely day

Micah January 23, 2025 at 4:17 am #

REPLY

Hi, I was wondering what adjustments I should make if my dataset contains multiple features and I want the target value to be the last feature of the dataset.

James Carmichael January 23, 2025 at 10:00 am #

REPLY

Hi Micah…When working with an LSTM for time series prediction in PyTorch and your dataset contains multiple features where the target value is the last feature, you can adjust your dataset preparation and model design to account for the multi-feature input and ensure the correct feature is used as the target. Here's a step-by-step guide:

—

1. **Prepare the Dataset**
You need to split your dataset into input features (X) and the target value (y), ensuring the last feature is used as the target.

Example:
Assume your dataset is a NumPy array or a pandas DataFrame with shape `(n_samples, n_features)`:

```
python
import numpy as np
import pandas as pd

# Example dataset
data = np.random.rand(1000, 5) # 5 features
df = pd.DataFrame(data, columns=['feature1', 'feature2', 'feature3', 'feature4', 'target'])

# Separate input features (X) and target (y)
X = df.iloc[:, :-1].values # All features except the last
y = df.iloc[:, -1].values # Last feature as the target
```

—

2. **Create Sequences**
Time series models require sequential data. Create sequences of input features (X) and corresponding targets (y) for each sequence.

Example:

```
python
def create_sequences(X, y, sequence_length):
sequences = []
targets = []
for i in range(len(X) - sequence_length):
seq = X[i:i+sequence_length]
target = y[i+sequence_length]
sequences.append(seq)
```

```
targets.append(target)
return np.array(sequences), np.array(targets)

# Parameters
sequence_length = 10
X_seq, y_seq = create_sequences(X, y, sequence_length)

# X_seq shape: (n_sequences, sequence_length, n_features)
# y_seq shape: (n_sequences,)
```

—

3. **Convert to PyTorch Tensors**
Convert the prepared sequences and targets into PyTorch tensors:

```
python
import torch
from torch.utils.data import TensorDataset, DataLoader

# Convert to tensors
X_tensor = torch.tensor(X_seq, dtype=torch.float32)
y_tensor = torch.tensor(y_seq, dtype=torch.float32)

# Create DataLoader for batch processing
dataset = TensorDataset(X_tensor, y_tensor)
data_loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

—

4. **Modify the LSTM Model**
Ensure your LSTM model is configured to handle multiple input features and outputs a single target value.

Example LSTM Model:

```
python
import torch.nn as nn

class LSTMTimeSeries(nn.Module):
def __init__(self, input_dim, hidden_dim, num_layers, output_dim):
super(LSTMTimeSeries, self).__init__()
self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers, batch_first=True)
self.fc = nn.Linear(hidden_dim, output_dim)

def forward(self, x):
# x shape: (batch_size, sequence_length, input_dim)
lstm_out, _ = self.lstm(x) # lstm_out shape: (batch_size, sequence_length, hidden_dim)
out = self.fc(lstm_out[:, -1, :]) # Take the output from the last time step
return out

# Model parameters
input_dim = X_seq.shape[2] # Number of features
hidden_dim = 64
num_layers = 2
output_dim = 1 # Single target value

# Initialize model
model = LSTMTimeSeries(input_dim, hidden_dim, num_layers, output_dim)
```

—

5. **Train the Model**
Use a suitable loss function (e.g., `nn.MSELoss` for regression) and an optimizer to train your model.

Example Training Loop:

```
python
# Loss and optimizer
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Training loop
num_epochs = 20
for epoch in range(num_epochs):
for inputs, targets in data_loader:
# Forward pass
outputs = model(inputs)
loss = criterion(outputs.squeeze(), targets)

# Backward pass and optimization
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

```
print(f"Epoch {epoch+1}/{num_epochs}, Loss: {loss.item():.4f}")
```

—

Key Adjustments for Multi-Feature Input:
1. **Separate Target**: Ensure the target is separated from the input features during dataset preparation.
2. **Sequence Creation**: Generate sequences of the input features while aligning them with the target values.
3. **Model Input Dimension**: The `input_dim` of the LSTM model should match the number of input features.
4. **Output Dimension**: The final output layer (`nn.Linear`) should produce a single value corresponding to the target.

Micah January 28, 2025 at 3:55 pm #

REPLY

Thank you for your response. I was wondering if you would use the same method for plotting the data or a different method since we're now dealing with different shapes.

Manel March 29, 2025 at 8:58 am #

REPLY

Hi, I think I'm a bit confused. When you apply your trained model to X_test :

test_plot[train_size+lookback:len(timeseries)] = model(X_test)[:, -1, :]

I feel like this is not correct from a machine learning standard, because X_test is basically a bunch of points from your test set of your timeseries. So… it's basically data that you shouldn't have access to at this point, right ?

What I'm doing is : taking the last point from y_train, which is the last point from the train set of the timeseries, and then I iterate : I apply the model to this point and I get my first predicted point in the test set area. Then I apply the model again on this predicted point to get my next prediction. And so on. Of course, my prediction is quite off, but I feel like it makes sense not to use the data of the test set at all, right ?

Then my thought is to try a larger look_back, so the model has more data to make a prediction.

Please let me know if I'm wrong in my logic. Thanks ! 🙂

James Carmichael March 30, 2025 at 4:15 am #

REPLY

Hi Manel…You're asking a **really great question**, and your thinking is actually quite aligned with good machine learning practices, especially when it comes to **time series forecasting**.

Let's break it down to clarify:

—

🔁 The Code You Mentioned:
```
python
test_plot[train_size+lookback:len(timeseries)] = model(X_test)[:, -1, :]
```

This line *does* indeed apply the trained model to `X_test`, which is a **sliding window of known sequences from the test set**. That means you're using actual test data (even if just input sequences) to get the predictions. So yes — **this is not true forecasting**. This is more like **evaluating performance on held-out known data** (sometimes called teacher-forcing), not simulating real-world conditions where future data is unknown.

—

✅ Your Approach (Autoregressive Forecasting):
What you're doing — where you:
1. Take the last `look_back` window from training data,
2. Predict the next time step,
3. Append that prediction to your input window,
4. Slide the window forward using your own predicted point as the new input,

This is **exactly how you'd use the model in production**: predicting *strictly* from past observed (or previously predicted) values. This is called **recursive** or **autoregressive forecasting**.

You're spot-on that this better simulates how forecasting is done in practice — where you don't get to "peek" at actual future values.

—

📏 Why the Difference in Results?
The model is trained on clean sequences where the inputs are true past values. When you start feeding in **predicted values** instead of real ones, **error compounds** — a problem known as **exposure bias** or **error accumulation**. This is why your predictions "drift" or look worse than when evaluated with `X_test`.

—

🧠 How to Improve Recursive Forecasting:
1. **Train with Noise / Scheduled Sampling**: During training, sometimes replace ground truth inputs with predicted ones. This teaches the model to

deal with its own errors.
2. **Use Sequence-to-Sequence LSTM**: Predict multiple time steps at once (many-to-many), rather than one at a time.
3. **Try a Transformer or Temporal Convolutional Network**: These may handle longer dependencies better than standard LSTMs.
4. **Increase Lookback Window**: Like you suggested — giving the model more context can help it make more stable predictions.

—

🧪 When Is `X_test` Usage OK?
It's okay to use `X_test` when:
– You're **evaluating** the model's performance (e.g., RMSE over the test set).
– You want to see how well the model generalizes on **known sequences** from a holdout set.
But you should **not** use it during a true forecasting simulation.

—

✅ Summary:
– You're right — using `X_test` in forecasting is not realistic.
– Your autoregressive method is correct for real forecasting.
– It's harder (and more honest), but better simulates reality.
– Larger `look_back` may help; also consider improvements like sequence prediction or noise-aware training.

—

Ron December 26, 2025 at 11:59 pm #

REPLY

Hello, thank you for this great tutorial!

I am wondering what would be the state-of-the-art result for this dataset?

Also, is there a model that can perfectly predict the test portion of the curve?
Looking at the curve as a human, I'm not sure I'd be able to perfectly predict the curve, since it is slightly noisy wrt the training part.

Happy to hear your thoughts aobut this.

James Carmichael December 29, 2025 at 10:12 am #

REPLY

Hi Ron…You are very welcome! Many have achieved success with hybrid CNN+LSTM models.

https://machinelearningmastery.com/start-here/#deep_learning_time_series

DS January 2, 2026 at 9:42 am #

REPLY

Thank you for all your hard work and time to put this together. I'm new to LTSM and I am trying to understand what is being forecasted. In the code sample:

```
with torch.no_grad():
# shift train predictions for plotting
train_plot = np.ones_like(timeseries) * np.nan
y_pred = model(X_train)
y_pred = y_pred[:, -1, :]
train_plot[lookback:train_size] = model(X_train)[:, -1, :]
# shift test predictions for plotting
test_plot = np.ones_like(timeseries) * np.nan
test_plot[train_size+lookback:len(timeseries)] = model(X_test)[:, -1, :]
# plot
plt.plot(timeseries)
plt.plot(train_plot, c='r')
plt.plot(test_plot, c='g')
plt.show()
```

I can see the statement:

```
y_pred = model(X_train)
```

But you are not plotting the prediction. In addition, you are just plotting over what is already there. I am not sure I follow in terms of where the prediction is taking place because if you already have the test data and are forecasting the test data I am not sure where in the exercise a prediction is being made and where to obtain that result. I am asking to understand and learn how the prediction process takes place.

James Carmichael January 2, 2026 at 9:47 am #

REPLY

Hi DS…More examples are found here: https://machinelearningmastery.com/start-here/#deep_learning_time_series

Please review them and reach back out with any questions.

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT

Welcome!
I'm *Jason Brownlee* PhD
and I **help developers** get results with **machine learning**.
Read more

Never miss a tutorial:

Picked for you:

PyTorch Tutorial: How to Develop Deep Learning Models with Python

LSTM for Time Series Prediction in PyTorch

Deep Learning with PyTorch (9-Day Mini-Course)

Calculating Derivatives in PyTorch


Building a Regression Model in PyTorch